

บทที่ 1 บทนำ

1.1 ประเภทของภาษาคอมพิวเตอร์

ภาษาคอมพิวเตอร์สามารถแบ่งออกได้หลายประเภทตามลักษณะการทำงานหรือระดับความใกล้เคียงกับภาษาเครื่อง (Machine Language) ดังนี้

1.1.1 ภาษาระดับต่ำ (Low-Level Languages)

ภาษากลุ่มนี้เป็นภาษาที่ใกล้เคียงกับภาษาเครื่องมากที่สุด คอมพิวเตอร์สามารถเข้าใจและประมวลผลได้โดยตรงหรือมีการแปลเพียงเล็กน้อย ทำให้การทำงานรวดเร็วและมีประสิทธิภาพสูง แต่การเขียนและการอ่านโค้ดทำได้ยากสำหรับมนุษย์

ภาษาเครื่อง (Machine Language) เป็นภาษาที่คอมพิวเตอร์เข้าใจได้โดยตรง ประกอบด้วยชุดคำสั่งที่เป็นรหัสเลขฐานสอง

ภาษาแอสเซมบลี (Assembly Language) เป็นภาษาระดับต่ำที่พัฒนาขึ้นมาเพื่อให้มนุษย์อ่านและเขียนได้ง่ายขึ้นกว่าภาษาเครื่อง โดยใช้คำสั่งที่เป็นสัญลักษณ์ (Mnemonics) แทน ชุดคำสั่งเลขฐานสอง ซึ่งจะถูกแปลเป็นภาษาเครื่องโดยโปรแกรม Assembler ก่อนที่คอมพิวเตอร์ จะนำไปประมวลผล

1.1.2 ภาษาระดับสูง (High-Level Languages)

ภาษากลุ่มนี้ถูกออกแบบมาเพื่อให้มนุษย์สามารถเขียนและอ่านได้ง่ายขึ้น มีโครงสร้างคล้ายภาษาอังกฤษ ทำให้การพัฒนาโปรแกรมทำได้รวดเร็วและง่ายต่อการบำรุงรักษา แต่จะต้องผ่านกระบวนการแปล (Compile หรือ Interpret) ให้เป็นภาษาเครื่องก่อนคอมพิวเตอร์จะนำไปทำงาน ตัวอย่างภาษาระดับสูงที่ เช่น Python, Java, C/C++, C#

1.2 ตัวแปลภาษาคอมพิวเตอร์ (Translator)

ตัวแปลภาษาคอมพิวเตอร์ (Translator) คือ โปรแกรมคอมพิวเตอร์ที่ทำหน้าที่แปลงชุดคำสั่งที่มนุษย์เขียนขึ้น (Source Code) ให้อยู่ในรูปแบบที่คอมพิวเตอร์เข้าใจและสามารถนำไปประมวลผลได้ (Machine Code หรือ Object Code) โดยทั่วไปแล้ว ภาษาที่มนุษย์เขียนขึ้นจะอยู่ในรูปแบบภาษาโปรแกรมระดับสูง (High-level Language) ซึ่งมีความใกล้เคียงกับภาษามนุษย์ ทำให้เขียนและทำความเข้าใจได้ง่ายกว่า ในขณะที่คอมพิวเตอร์จะเข้าใจเฉพาะภาษาเครื่อง (Machine Language) ซึ่งเป็นชุดคำสั่งเลขฐานสองเท่านั้น

ตัวแปลภาษาคอมพิวเตอร์สามารถแบ่งออกได้เป็น 3 ประเภทหลัก ๆ ดังนี้

1.2.1 คอมไพเลอร์ (Compiler) จะอ่าน Source Code ทั้งหมดของโปรแกรมตั้งแต่ต้นจนจบ แล้วทำการแปลเป็น Machine Code หรือ Object Code ทั้งหมดในคราวเดียว หากไม่พบข้อผิดพลาดใด ๆ ก็ sẽสร้างไฟล์ที่สามารถนำไปรันได้ (Executable File) ขึ้นมา ดังนั้นจะทำงานได้เร็ว เนื่องจากจะทำการแปลคำสั่งทั้งหมดในครั้งเดียว แล้วจึงทำงานตามคำสั่งของโปรแกรมในภายหลัง แต่เมื่อเกิดข้อผิดพลาดขึ้นจะตรวจสอบหาข้อผิดพลาดได้ยาก เพราะทำการแปลคำสั่งทีเดียวทั้งโปรแกรม

1.2.2 อินเทอร์พรีเตอร์ (Interpreter) จะอ่าน Source Code ทีละบรรทัด แปลและประมวลผลคำสั่งนั้นทันที โดยไม่สร้างไฟล์ Executable ขึ้นมา ทำให้หาข้อผิดพลาดของโปรแกรมได้ง่าย เนื่องจากแปลคำสั่งทีละบรรทัด สามารถสั่งให้โปรแกรมทำงานเฉพาะจุดได้ ไม่เสียเวลารอการแปลคำสั่งเป็นเวลานาน แต่การทำงานจะช้ากว่าโปรแกรมที่ใช้ Compiler เนื่องจากต้องทำการแปลและประมวลผลทีละบรรทัดทุกครั้งที่รัน

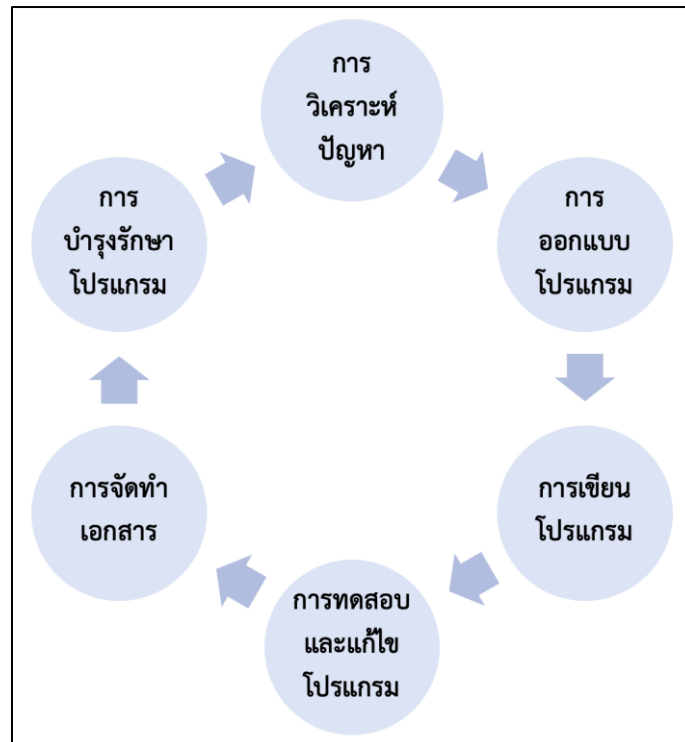
แอสเซมเบลอร์ (Assembler) ทำหน้าที่แปลภาษาแอสเซมบลี (Assembly Language) ซึ่งเป็นภาษาโปรแกรมระดับต่ำ (Low-level Language) ที่ใช้สัญลักษณ์ (Mnemonics) แทนชุดคำสั่งภาษาเครื่องโดยตรง ให้เป็นภาษาเครื่อง (Machine Code)

1.3 วงจรการพัฒนาโปรแกรม (Program Development Life Cycle: PDLC)

วงจรการพัฒนาโปรแกรม คือ กระบวนการที่เป็นระบบและมีขั้นตอนในการพัฒนาโปรแกรม หรือซอฟต์แวร์ ตั้งแต่จุดเริ่มต้นของการกำหนดความต้องการไปจนถึงการส่งมอบและบำรุงรักษา เป้าหมายหลักของ PDLC คือเพื่อให้แน่ใจว่าซอฟต์แวร์ที่พัฒนาขึ้นมีคุณภาพ ตอบสนองความต้องการของผู้ใช้งานได้อย่างมีประสิทธิภาพ

Program Development Life Cycle (PDLC) ประกอบด้วยงาน 6 อย่างดังนี้

1. การวิเคราะห์ปัญหา (Problem analysis)
2. การออกแบบโปรแกรม (Program Design)
3. การเขียนโปรแกรม (Programming/Implementation)
4. การทดสอบและแก้ไขโปรแกรม (Testing and Debugging)
5. การจัดทำเอกสาร (Programming Documentation)
6. การบำรุงรักษาโปรแกรม (Maintenance)



รูป 1 วงจรการพัฒนาโปรแกรม (Program Development Life Cycle: PDLC)

1. การวิเคราะห์ปัญหา (Problem analysis)

คือการศึกษาและทำความเข้าใจปัญหาอย่างละเอียด เพื่อให้ได้ข้อมูลที่เพียงพอสำหรับการออกแบบโปรแกรม เป็นขั้นตอนที่สำคัญของการเขียนโปรแกรม เป็นการวิเคราะห์ลักษณะของงาน ความต้องการ รายละเอียดปัญหาของงานนั้น ๆ วิเคราะห์สิ่งที่โจทย์ต้องการ ตัวแปรที่ใช้ รูปแบบผลลัพธ์ (Output) ข้อมูลนำเข้า (Input) วิธีการประมวลผล (Process) ระบุฟังก์ชันการทำงานที่จำเป็น (Functional Requirements) และข้อกำหนดที่ไม่ใช่ฟังก์ชันการทำงาน (Non-functional Requirements) เช่น ประสิทธิภาพ ความปลอดภัย

2. การออกแบบโปรแกรม (Program Design)

คือการแปลงข้อกำหนดความต้องการที่ได้จากขั้นตอนการวิเคราะห์ปัญหา (Problem analysis) ให้เป็นแผนงานที่ชัดเจนสำหรับการสร้างโปรแกรม เช่น ออกแบบฐานข้อมูล (Database Design) ออกแบบส่วนติดต่อผู้ใช้งาน (User Interface - UI) ออกแบบอัลกอริทึม (Algorithm Design) ด้วยผังงาน (Flowchart) หรือ Pseudocode สำหรับแต่ละส่วนของโปรแกรม

อัลกอริทึม (Algorithm) คือ ชุดของขั้นตอนหรือลำดับวิธีการทำงานที่ชัดเจนและเป็นระบบ เพื่อใช้ในการแก้ปัญหาใดปัญหาหนึ่งให้บรรลุเป้าหมายที่กำหนดไว้ โดยเป็นหัวใจสำคัญในการ

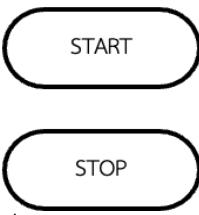
พัฒนาโปรแกรมคอมพิวเตอร์และระบบงานต่าง ๆ เนื่องจากเป็นพื้นฐานในการแปลงแนวคิดไปสู่ชุดคำสั่งที่คอมพิวเตอร์สามารถเข้าใจและประมวลผลได้

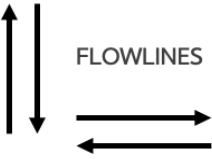
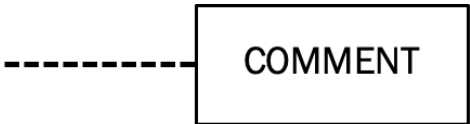
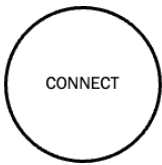
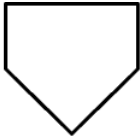
การออกแบบอัลกอริทึม (Algorithm Design) คือ ขั้นตอนการวางแผนเพื่อนำผลลัพธ์ที่ได้จากการวิเคราะห์ปัญหา มาจัดลำดับขั้นตอนและกระบวนการในการแก้ปัญหาอย่างเป็นระบบ ซึ่งการออกแบบนี้จะถูกแสดงออกมาในรูปแบบของอัลกอริทึม โดยมีวัตถุประสงค์เพื่อให้ง่ายต่อการทำความเข้าใจ การตรวจสอบความถูกต้อง และการนำไปเขียนเป็นโปรแกรมคอมพิวเตอร์ต่อไป เครื่องมือที่นิยมใช้ในการแสดงการออกแบบอัลกอริทึม ได้แก่

รหัสจำลอง (Pseudocode) เป็นรูปแบบการอธิบายขั้นตอนการทำงานของอัลกอริทึมที่ใช้ภาษาใกล้เคียงกับภาษามนุษย์ แต่มีการจัดโครงสร้างและคำศัพท์เฉพาะที่คล้ายคลึงกับภาษาโปรแกรมคอมพิวเตอร์ เพื่อให้เข้าใจง่ายและสามารถแปลงเป็นโค้ดจริงได้สะดวก

ผังงาน (Flowchart) เป็นแผนภาพที่ใช้สัญลักษณ์มาตรฐานต่าง ๆ เพื่อแสดงลำดับขั้นตอนการทำงาน การตัดสินใจ และทิศทางการไหลของข้อมูลในอัลกอริทึมอย่างเป็นระบบ ช่วยให้มองเห็นภาพรวมและตรรกะของกระบวนการแก้ปัญหาได้ชัดเจน

ตาราง 1 ตัวอย่างสัญลักษณ์ของผังงาน

สัญลักษณ์	ความหมาย
 สัญลักษณ์จุดเริ่มต้นและจุดจบของโปรแกรม	ใช้แทนสัญลักษณ์จุดเริ่มต้นการทำงานของโปรแกรมและจุดสิ้นสุดการทำงานของโปรแกรม แทนสัญลักษณ์ด้วยรูปสี่เหลี่ยมมุมมน
 สัญลักษณ์การประมวลผล	ใช้แทนการประมวลผลและสูตรคำนวณต่าง ๆ แทนสัญลักษณ์ด้วยรูปสี่เหลี่ยมผืนผ้า
 สัญลักษณ์อินพุตและเอาต์พุต	ใช้สำหรับรับข้อมูลนำเข้าหรือแสดงผลลัพธ์ โดยไม่ได้ระบุชนิดของอุปกรณ์ที่ชัดเจน แทนสัญลักษณ์ด้วยรูปสี่เหลี่ยมขนมเปียกปูน

สัญลักษณ์	ความหมาย
 สัญลักษณ์เส้นโฟลว์กำหนดทิศทางข้อมูล	ใช้สำหรับแสดงทิศทางการไหลของข้อมูลในโปรแกรม แทนสัญลักษณ์เส้นพร้อมหัวลูกศร
 สัญลักษณ์การตัดสินใจ	ใช้ตรวจสอบเงื่อนไขเพื่อตัดสินใจทางเลือก
 สัญลักษณ์ทำงานเป็นรอบ	ใช้สำหรับการวนซ้ำเพื่อทำงานตามจำนวนรอบ
 สัญลักษณ์คำอธิบายเพิ่มเติม	ใช้อธิบายรายละเอียดเกี่ยวกับกิจกรรมนั้น ๆ
 สัญลักษณ์แสดงจุดเชื่อมโยงภายในหน้าเดียวกัน	ใช้เป็นจุดเชื่อมโยงแผนภาพที่อยู่ในหน้าเดียวกัน หรือใช้เป็นจุดให้เส้นโฟลว์มาบรรจบกัน แทนสัญลักษณ์ด้วยวงกลม
 OFF-PAGE CONNECTOR สัญลักษณ์แสดงจุดเชื่อมโยงที่อยู่คนละหน้า	ใช้เชื่อมโยงเข้ากับผังงานที่อยู่คนละหน้า

สัญลักษณ์	ความหมาย
 สัญลักษณ์แสดงผลลัพธ์ทางเครื่องพิมพ์	ใช้สำหรับแสดงผลลัพธ์ทางเครื่องพิมพ์
 สัญลักษณ์โปรแกรมย่อย	ใช้สำหรับแทนการเรียกใช้กลุ่มงานหรือโปรแกรมย่อย

3. การเขียนโปรแกรม (Programming/Implementation)

เป็นการเปลี่ยนขั้นตอนวิธีที่ได้ออกแบบไว้ ให้อยู่ในภาษาคอมพิวเตอร์ (Programming language) ภาษาใดภาษาหนึ่ง การเลือกใช้ภาษาคอมพิวเตอร์ใดนั้น ขึ้นอยู่กับลักษณะและประเภทของงาน ชีตความสามารถและข้อจำกัดของภาษา และความถนัดความชำนาญของผู้เขียนโปรแกรม

4. การทดสอบและแก้ไขโปรแกรม (Testing and Debugging)

เป็นขั้นตอนการตรวจสอบว่าโปรแกรมทำงานได้อย่างถูกต้อง มีคุณภาพ และเป็นไปตามข้อกำหนดหรือไม่ ทั้งในส่วนข้อไวยากรณ์ (Syntax Error) และตรรกะ (Logic Error) การทดสอบสามารถแบ่งออกได้หลายประเภทตามมุมมองและวัตถุประสงค์ของการทดสอบ เช่น ทดสอบแต่ละส่วนของโปรแกรม (Unit Testing) ทดสอบการทำงานร่วมกันของส่วนต่างๆ (Integration Testing) ทดสอบระบบโดยรวม (System Testing) ทดสอบการยอมรับของผู้ใช้งาน (User Acceptance Testing - UAT)

5. การจัดทำเอกสาร (Programming Documentation)

กระบวนการรวบรวม จัดระเบียบ และเขียนข้อมูลที่เกี่ยวข้องกับโปรแกรมหรือซอฟต์แวร์ ตั้งแต่แนวคิดเริ่มต้นไปจนถึงการใช้งานและการบำรุงรักษาในระยะยาว จุดประสงค์หลักคือเพื่อให้ข้อมูลที่เป็นประโยชน์แก่ผู้มีส่วนเกี่ยวข้อง ไม่ว่าจะเป็นโปรแกรมเมอร์คนอื่น ผู้จัดการโครงการ ผู้ใช้งาน หรือแม้แต่ตัวผู้พัฒนาเองในอนาคต

ข้อดีของการจัดทำเอกสารที่ดี

1. ช่วยทำความเข้าใจ การมีเอกสารช่วยให้ผู้ที่เกี่ยวข้องเข้าใจระบบได้ง่ายขึ้น ไม่ว่าจะเป็นวิธีการทำงาน เหตุผลในการตัดสินใจออกแบบ หรือวิธีการใช้งาน
2. ช่วยบำรุงรักษา เอกสารทำให้การแก้ไข ปรับปรุง หรือขยายระบบในอนาคตทำได้ง่ายขึ้น เพราะมีข้อมูลอ้างอิงที่ชัดเจน
3. ช่วยถ่ายทอดความรู้ ช่วยให้องค์ความรู้เกี่ยวกับระบบไม่ติดอยู่กับบุคคลใดบุคคลหนึ่ง หากโปรแกรมเมอร์เดิมไม่อยู่แล้ว คนใหม่ก็สามารถเรียนรู้ได้จากเอกสาร
4. ช่วยสื่อสาร เอกสารเป็นเครื่องมือสำคัญในการสื่อสารระหว่างทีมงาน ผู้ใช้งานและผู้มีส่วนได้ส่วนเสีย
5. ช่วยควบคุมคุณภาพ การมีเอกสารที่ดีบ่งชี้ถึงกระบวนการพัฒนาที่มีคุณภาพและเป็นระเบียบ

6. การบำรุงรักษาโปรแกรม (Maintenance)

เป็นกระบวนการตรวจสอบดูแลแก้ไขข้อผิดพลาดที่เกิดขึ้นหลังจากนำโปรแกรมไปใช้งานจริง อัปเดตและพัฒนาปรับปรุงประสิทธิภาพของโปรแกรม เพิ่มฟังก์ชันใหม่ ๆ หรือปรับปรุงฟังก์ชันที่มีอยู่ เพิ่มเติมตามความต้องการที่เปลี่ยนแปลงไป

1.4 ประวัติความเป็นมาของภาษาไพธอน

ภาษา Python ถูกสร้างขึ้นโดย Guido van Rossum ในช่วงปลายทศวรรษ 1980 และเปิดตัวครั้งแรกในปี 1991 ชื่อ "Python" มาจากรายการตลกของอังกฤษที่ชื่อ "Monty Python's Flying Circus" ซึ่ง Guido ชื่นชอบ

การออกแบบหลักของ Python มุ่งเน้นไปที่ความสามารถในการอ่านโค้ด (Code Readability) ซึ่งเป็นเหตุผลสำคัญที่ทำให้ Python เป็นภาษาที่เหมาะสมสำหรับผู้เริ่มต้น และได้รับความนิยมอย่างกว้างขวางในหลายสาขา

พัฒนาการของ Python มีมายาวนานและต่อเนื่อง ในปัจจุบัน Python 3.x เป็นเวอร์ชันมาตรฐานที่ใช้กันทั่วไปและแนะนำสำหรับการเรียนรู้และการพัฒนาใหม่ ๆ Python ได้รับความนิยมรับว่าเป็นหนึ่งในภาษาโปรแกรมที่ได้รับความนิยม และมีการใช้งานอย่างแพร่หลายในหลากหลายงาน เช่น

- เว็บแอปพลิเคชัน (เช่นด้วยเฟรมเวิร์ค Django และ Flask)

- ปัญญาประดิษฐ์ (AI)
- การประมวลผลภาพ (Visualization)
- การเรียนรู้ของเครื่อง (Machine Learning)
- การวิเคราะห์ข้อมูล (Data Analysis)

1.4.1 จุดเด่นของภาษา Python

1. คุณสมบัติ Portable Language

Python โดดเด่นด้วยคุณสมบัติการเป็น Portable Language ซึ่งหมายถึงความสามารถในการรันโค้ดบนแพลตฟอร์มคอมพิวเตอร์ที่หลากหลายโดยไม่ต้องแก้ไขโค้ดอย่างมีนัยสำคัญ (แนวคิด "Write Once, Run Anywhere") โค้ด Python ที่พัฒนาบนระบบปฏิบัติการหนึ่ง เช่น Windows สามารถนำไปประมวลผลบน macOS หรือ Linux ได้อย่างรวดเร็ว ส่งผลให้กระบวนการพัฒนามีความยืดหยุ่นสูง

2. สนับสนุน Graphical User Interface (GUI)

Python มีไลบรารีและเฟรมเวิร์กจำนวนมากที่ใช้สำหรับการพัฒนาแอปพลิเคชันที่มีส่วนติดต่อผู้ใช้แบบกราฟิก (GUI) ทำให้สามารถสร้างโปรแกรมที่มีองค์ประกอบกราฟิก เช่น หน้าต่าง ปุ่ม และเมนู ได้อย่างมีประสิทธิภาพ เช่น Tkinter

3. ภาษาแบบ Dynamically-Typed

Python จัดเป็นภาษาแบบ Dynamically-Typed ซึ่งหมายความว่าไม่จำเป็นต้องประกาศชนิดข้อมูล (เช่น integer, string) ของตัวแปรล่วงหน้า ภาษาจะกำหนดชนิดข้อมูลโดยอัตโนมัติ ณ เวลาที่รันโปรแกรมตามค่าที่กำหนดให้ คุณสมบัตินี้ช่วยเพิ่มความเร็วและความยืดหยุ่นในการเขียนโค้ด อาทิเช่น การสามารถกำหนดค่า $x = 10$ และปรับเปลี่ยนเป็น $x = \text{"Hello"}$ ได้ในภายหลัง โดยไม่ต้องดำเนินการประกาศชนิดข้อมูลเพิ่มเติม ซึ่งช่วยลดเวลาการพัฒนาและทำให้โค้ดกระชับ

4. ความสามารถในการบูรณาการกับภาษาอื่น

Python มีความสามารถในการทำงานร่วมกับภาษาโปรแกรมอื่น ๆ ได้อย่างมีประสิทธิภาพ อาทิ การเรียกใช้โค้ดที่พัฒนาด้วยภาษา C, C++, Java, หรือ Fortran จากโค้ด Python

ซึ่งเป็นประโยชน์อย่างยิ่งในการใช้ประโยชน์จากไลบรารีที่มีอยู่เดิม หรือเพื่อเพิ่มประสิทธิภาพในส่วนที่ต้องการความเร็วในการประมวลผลสูง

5. เป็น Open Source

Python เป็นภาษาแบบ Open Source ทำให้ผู้ใช้งานสามารถดาวน์โหลด ใช้งาน ศึกษา และปรับปรุงแก้ไขโค้ดได้อย่างอิสระโดยไม่มีค่าใช้จ่าย คุณสมบัตินี้เป็นประโยชน์อย่างยิ่งต่อบุคคลทั่วไป สถาบันการศึกษา และองค์กรทุกขนาด เนื่องจากช่วยลดต้นทุนในการพัฒนาซอฟต์แวร์

6. โครงสร้างไวยากรณ์ที่เข้าใจง่าย

Python ได้รับการออกแบบให้มีไวยากรณ์ที่เรียบง่าย ชัดเจน และใกล้เคียงกับภาษาอังกฤษ การใช้การเยื้อง (Indentation) แทนวงเล็บปีกกา {} เพื่อกำหนดขอบเขตของบล็อกโค้ด ช่วยให้โค้ดมีความเป็นระเบียบและอ่านง่ายอย่างยิ่ง จึงเหมาะสำหรับผู้เริ่มต้นเรียนรู้การเขียนโปรแกรม

7. สนับสนุนการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming - OOP)

Python สนับสนุนแนวคิดการเขียนโปรแกรมเชิงวัตถุ (OOP) ได้อย่างสมบูรณ์ ซึ่งช่วยให้การออกแบบและพัฒนาซอฟต์แวร์ขนาดใหญ่มีความเป็นระเบียบ สามารถนำโค้ดกลับมาใช้ซ้ำได้ (Reusability) และสะดวกต่อการบำรุงรักษา นักพัฒนาสามารถสร้าง Class และ Object เพื่อจำลองสิ่งต่าง ๆ ในโลกแห่งความเป็นจริงได้

8. มีไลบรารีและเฟรมเวิร์กจำนวนมาก

Python มีชุมชนนักพัฒนาขนาดใหญ่ ส่งผลให้มีไลบรารี (Libraries) และเฟรมเวิร์ก (Frameworks) ที่ได้รับการพัฒนาอย่างต่อเนื่องและหลากหลาย ครอบคลุมการใช้งานเกือบทุกแขนง เช่น

- การพัฒนาเว็บ: Django, Flask
- การพัฒนาเกม: Pygame
- ประมวลผลภาพและกราฟิก: Pillow, OpenCV
- ปัญญาประดิษฐ์ (AI)/การเรียนรู้ของเครื่อง (ML): TensorFlow, Scikit-learn
- วิทยาศาสตร์ข้อมูล (Data Science): NumPy, Pandas, Matplotlib

9. การรองรับการทำงานกับฐานข้อมูล

Python มีโมดูลและไลบรารีจำนวนมากที่อำนวยความสะดวกในการเชื่อมต่อและทำงานกับฐานข้อมูลประเภทต่าง ๆ ได้อย่างง่ายดาย ไม่ว่าจะเป็นฐานข้อมูลเชิงสัมพันธ์ (Relational Databases) เช่น MySQL, PostgreSQL, SQLite หรือฐานข้อมูล NoSQL เช่น MongoDB คุณสมบัตินี้ทำให้ Python เป็นตัวเลือกที่เหมาะสมสำหรับการพัฒนาแอปพลิเคชันที่ต้องการจัดการข้อมูลปริมาณมาก

1.5 เครื่องมือที่ใช้ในการโปรแกรม

IDE ย่อมาจาก Integrated Development Environment เป็นชุดโปรแกรมที่รวบรวมเครื่องมือที่จำเป็นสำหรับนักพัฒนาซอฟต์แวร์ไว้ในที่เดียว เพื่อช่วยให้กระบวนการเขียนโค้ดคอมไพล์ ดีบั๊ก และทดสอบโปรแกรมเป็นไปอย่างราบรื่นและมีประสิทธิภาพมากขึ้น

IDE สำหรับการเขียนโปรแกรมด้วยภาษาไพธอนมีหลายประเภท ทั้งแบบที่สามารถติดตั้งใช้งานบนเครื่องคอมพิวเตอร์ (Offline) และแบบที่สามารถใช้งานผ่านเว็บเบราว์เซอร์ได้เลย (Online) ซึ่งเหมาะกับการใช้งานที่แตกต่างกันไป

ตาราง 2 ตัวอย่าง IDE สำหรับการเขียนโปรแกรมด้วยภาษาไพธอน

แบบติดตั้งบนเครื่อง (Offline)	แบบใช้งานบนเว็บเบราว์เซอร์ (Online)
- Python IDLE www.python.org - Thonny https://thonny.org/ - PyCharm www.jetbrains.com/pycharm - Visual Studio Code (VS Code) https://code.visualstudio.com/ - Spyder www.anaconda.com/download	- Google Colaboratory (Colab) https://colab.research.google.com/ (ต้องใช้บัญชี gmail) - Replit https://replit.com/ - Programiz Online Python Compiler https://www.programiz.com/python-programming/online-compiler/ - OnlineGDB https://www.onlinegdb.com/online_python_compiler

1.5.1 Google Colaboratory



Google Colaboratory หรือ Colab คือ IDE ที่ทำงานบนคลาวด์ ซึ่งได้รับการพัฒนาโดย Google Research ที่สามารถเขียนและรันโค้ด Python ได้โดยตรงจากเว็บเบราว์เซอร์ ช่วยให้ผู้ใช้สามารถพัฒนาโปรแกรมได้โดยไม่ต้องติดตั้งซอฟต์แวร์หรือกำหนดค่าสภาพแวดล้อมใดๆ บนเครื่องคอมพิวเตอร์ของตน

คุณสมบัติและประโยชน์หลักของ Colab

1. การเข้าถึงที่สะดวกและไม่มีค่าใช้จ่าย ผู้ใช้งานสามารถเข้าถึง Colab ได้อย่างง่ายดายผ่านบัญชี Google เป็นบริการฟรีที่ช่วยลดอุปสรรคในการเข้าถึงทรัพยากรการประมวลผลสำหรับการเรียนรู้และการพัฒนา
2. สภาพแวดล้อมที่ไม่ต้องติดตั้ง (Zero-Setup Environment) Colab มาพร้อมกับการติดตั้ง Python และไลบรารีที่จำเป็นจำนวนมากไว้ล่วงหน้า (Pre-installed) เช่น NumPy, Pandas, TensorFlow, PyTorch ช่วยให้ผู้ใช้สามารถเริ่มต้นเขียนและรันโค้ดได้ทันทีโดยไม่ต้องจัดการกับการติดตั้งและการกำหนดค่าที่ซับซ้อน
3. การประมวลผลประสิทธิภาพสูงด้วย GPU และ TPU หนึ่งในจุดเด่นที่สำคัญที่สุดของ Colab คือการที่ผู้ใช้งานสามารถเข้าถึงหน่วยประมวลผลกราฟิก (GPU) และหน่วยประมวลผล Tensor (TPU) ได้ฟรี ทรัพยากรเหล่านี้มีความสำคัญอย่างยิ่ง สำหรับงานที่ต้องใช้การประมวลผลเชิงคำนวณสูง เช่น การฝึกโมเดลปัญญาประดิษฐ์ (Artificial Intelligence - AI) และการเรียนรู้เชิงลึก (Deep Learning) ซึ่งช่วยเร่งความเร็วในการคำนวณได้อย่างมีนัยสำคัญ
4. การบูรณาการกับ Google Drive Colab Notebooks จะถูกจัดเก็บโดยอัตโนมัติใน Google Drive ของผู้ใช้งาน ช่วยให้การจัดเก็บ การจัดการ และการเข้าถึงไฟล์ข้อมูลสำหรับโปรเจกต์ทำได้สะดวก
5. ความสามารถในการทำงานร่วมกัน (Collaboration) Colab รองรับการทำงานร่วมกันแบบเรียลไทม์ (Real-time Collaboration) คล้ายกับ Google Docs ผู้ใช้งานหลายคน

สามารถทำงานบน Colab Notebook เดียวกันได้พร้อมกัน ซึ่งเอื้อต่อการทำงานเป็นทีม และการสอน

6. รูปแบบ Notebook ที่เป็นโครงสร้าง (Structured Notebook Format) Colab ใช้รูปแบบ Jupyter Notebook ซึ่งประกอบด้วย "เซลล์โค้ด" (Code Cells) สำหรับการเขียนและรันโค้ด Python และ "เซลล์ข้อความ" (Text Cells) ที่รองรับภาษา Markdown สำหรับการใส่คำอธิบาย หัวข้อ และเอกสารประกอบ ช่วยให้การนำเสนอ ข้อมูลโค้ด และผลลัพธ์อยู่ในรูปแบบที่ชัดเจนและอ่านง่าย

กลุ่มผู้ใช้งานหลักของ Colab

1. นักเรียนและนักศึกษา ใช้เป็นแพลตฟอร์มในการเรียนรู้และฝึกฝนการเขียนโค้ด Python โดยไม่มีข้อจำกัดด้านฮาร์ดแวร์
2. นักวิทยาศาสตร์ข้อมูลและนักวิจัย ใช้สำหรับการวิเคราะห์ข้อมูล การสร้างแบบจำลองสถิติ และการพัฒนาโมเดล Machine Learning/Deep Learning
3. นักพัฒนา ใช้สำหรับการทดลองโค้ดอย่างรวดเร็ว (Prototyping) และการทำงานร่วมกันในโปรเจกต์ต่าง ๆ